Ouroboros 安全评估 Agent：架构全量剖析与失败模式深度报告

代码级详细版（外援裁决用）• v27 实测快照

Ouroboros 项目组

2026 年 10 月 30 日

摘要

本报告面向外部架构顾问，提供 Ouroboros 观测-推理循环 Agent（GateMesh v1）的**代码级**架构剖析：全部数据结构定义、协议语法、执行时序、代码坐标（文件: 行号）、33 场实测的失败案例原文摘录、8 轮修复链的每次失效原因，以及由此推导的 v2 结构判断。核心矛盾一句话：*模型在子循环内解题质量高，在父循环编排上系统性失败*——v19 换旗舰模型后 11 个版本零新增解题，2 个成功解题均来自早期 flash 模型会话。文末列出五个需要裁决的架构问题，每个附我们的备选方案与倾向。

目录

1 项目定位与设计哲学 **3**
- 1.1 定位 . . . 3
- 1.2 设计哲学（用户定调，不可违背） . . . 3
- 1.3 评测环境契约（TSec Benchmark） . . . 3

2 架构全量剖析（GateMesh v1） **3**
- 2.1 五层结构与代码坐标 . . . 3
- 2.2 核心数据结构（原文） . . . 3
  - 2.2.1 观测原语（nativetools/ObserveSpec 与 ObserveResult） . . . 3
  - 2.2.2 黑板事实（api/BlackboardFact） . . . 5
  - 2.2.3 子循环状态（api/subLoopState） . . . 5
- 2.3 动作协议（ACTION 文本协议，87 处引用） . . . 6
- 2.4 chat 主流程时序（ai_chat.go:96 handleAIChatPost） . . . 6
- 2.5 子循环生命周期（subloop.go:82/247/487） . . . 6
- 2.6 driver 编排（Python，benchmark/） . . . 7
- 2.7 Docker 形态 . . . 7

3 实测失败模式（FM-1 至 FM-5，含案例原文） **7**
- 3.1 FM-1：分析瘫痪（未解决，最高优先级） . . . 7
- 3.2 FM-2：格式执行失败（三代守卫，治标） . . . 8

3.3 FM-3：重复获取（信息经济学失衡） 8
3.4 FM-4：指令二义性死锁 8
3.5 FM-5：环境层（已驯服） 9
3.6 成功侧记录（公平性） 9

4 关键不对称与理论推导 **9**
4.1 三个结构性判断（v2 推导基础） 9
4.2 v2 蓝图（已推导未实现） 9
4.3 待验证假设（下个靶场窗口） 9

5 给外援的五个裁决问题 **9**

附录 A：关键代码坐标速查 **10**

1 项目定位与设计哲学

1.1 定位

面向**合规自动化渗透测试**的安全评估 Agent。技术栈：Go 1.26 后端（222,134 行）+ React/-Electron 前端 + Docker 部署。测试资产：150 个 `_test.go`。知识库：79 个 MIT 红队技能文档蒸馏为 1,889 个配方超图节点。

1.2 设计哲学（用户定调，不可违背）

1. **观测-推理循环**：工具只做零判定观测（HTTP 原语拿原始响应：状态/头/时延/表单/链接/基线差分），结论一律由模型基于证据得出。工具内置任何" 漏洞判定" 都被视为污染。
2. **黑板仅作存储**：跨会话结论笔记 + 证据回链（`source_observations` 字段），不是推理引擎。用户明确否决了" 黑板算法" 调度（嫌其固化模型思维）。
3. **模板冻结**：nuclei 兼容模板引擎冻结在 16 个内置（生态兼容层），扩展只走外部目录。判定属于模型，不属于模板。
4. 检测/验证/凭据测试在范围内；exploitation/后渗透由 managed bundle 承担（平台边界）。

1.3 评测环境契约（TSec Benchmark）

- 63 题（easy 9 / medium 25 / hard 29），总分 23,000
- API：`GET /openapi/v1/challenges`（列题）、`POST .../start?unique_code=`（启动，返回 container_addr）、`POST .../submit`（JSON body 提交 flag）、`POST .../close`、`GET .../hint`（扣分）
- 认证：`BENCHMARK_TOKEN` 请求头；3 容器并发上限；VPN 内网直连（10.0.x.x）
- 题型分布：e1-x（WAF/边界绕过）、e2-x（沙箱逃逸）、e3-x（AV/外带）、a-x（Web 系统）、b-x（全链路渗透）、c-x（面板/服务）、d-x（云安全）、f1-x（内存安全）、f2-x（逆向）

2 架构全量剖析（GateMesh v1）

2.1 五层结构与代码坐标

2.2 核心数据结构（原文）

2.2.1 观测原语（nativetools/ObserveSpec 与 ObserveResult）

Listing 1: internal/nativetools/http_observe.go:28-68

```
type ObserveSpec struct {
    Method      string            `json:"method,omitempty"`
    Path        string            `json:"path"`
    Query       map[string]string `json:"query,omitempty"`
    Headers     map[string]string `json:"headers,omitempty"`
    Body        string            `json:"body,omitempty"`
    ContentType string            `json:"content_type,omitempty"`
```

层	组件	代码位置(行数)	职责与机制
L4 知识	配方超图	hypergraph.go (276)	1,889 节点各带 12 维前置/排除位掩码；实体状态 ∩ 前置 ≥ 70% 唤醒；注入 ≤ 1.5K 切片
	技能索引	kb_retrieval.go (234)	79 技能词法检索；CJK→ 英文技术词 30 组桥接
L3 纪律	观测蒸馏	observe_distill.go (138)	ObserveResult JSON → ≤600B 摘要（diff 优先，fail-open 400B 回退）
	模型路由	provider_generation.go	`CognitiveTier`: 1= 战术（OURO_TACTIC_MODEL），2= 旗舰（门控轮）；子循环默认 1
L2 四门	Scope	observe_scope.go + egress_gate.go (139)	OBSERVE 闸门 + EXECUTE 出站门（URL 正则提取 → infra 拒绝/enforce 白名单/exclusion 前缀）；根路径预检豁免
	Evidence	evidence_gate.go (162)	`ACTION:VERIFY` 重放观测 → 信号子串匹配 → 关联 fact E2→E3
	Progress	subloop.go:247 内	IGR：黑板事实数 5 轮零新增 → 掉头注入（每循环 ≤ 2 次）；DONE 无 flag 驳回一次
	Budget	driver (Python)	全局轮次硬顶 1500；子循环总预算 600
L1 传送带	BENCH	conveyor.go (419)	五操作 Go 直调平台；蒸馏输出（list 带活跃题详情）；审计 bench-log.jsonl
L0 状态脊	事件账本	eventlog/ (SQLite WAL)	EventFact/Intent/Hint/Parser 类型；幂等四阶段租约；毒丸隔离
	子循环注册表	subloop.go:623+	subloops.jsonl 持久化；重启恢复（running→cancelled）

表 1: 五层结构全量组件与代码坐标

```
    Baseline    bool              `json:"baseline,omitempty"` // 差分控制组
}

type ObserveResult struct  // 零判定的全量可观测结果
    URL string `json:"url"`
    Method string; Query map[string]string; Headers map[string]string
    ContentType string
    Status int; StatusText string
    ResponseHeaders map[string][]string
    SetCookies []string; Location string; RedirectChain []string
    BodySize int; BodySHA256 string; BodyPreview string
    ElapsedMS float64; Server string; XPoweredBy string
    Forms []ObserveForm      // 表单结构（action/字段名）
    Links []string           // 同源链接
    JSONKeys []string        // JSON 顶层键
    Diff *ObserveDiff        // 基线差分（Baseline=true 时）
}
```

关键语义：client 不跟随重定向（保留 3xx 供差分）；`{{env:VAR}}` 占位符在 URL/headers/-body/query/path 五处调用时展开（未设变量 fail-closed 报错）；观测结果 JSON 原文落盘 observations.jsonl，进对话的只有蒸馏摘要。

2.2.2 黑板事实（api/BlackboardFact）

Listing 2: internal/api/ai_chat.go:1159-1175

```
type BlackboardFact struct {
    ID            string   `json:"id"`           // fact-<UnixNano>-<rand4hex>
    Kind          string   `json:"kind"`
    Statement     string   `json:"statement"`    // 结论文本
    Source        string   `json:"source"`
    SourceObservations []string `json:"source_observations,omitempty"`
        // ^ 结论中引用的 obs-ID 自动提取为回链;
        //   sisyphus.ProcessedObservationIDs 据此核销观测积压
    EvidenceLevel int `json:"evidence_level,omitempty"`
        // ^ 0=笔记 2=Tentative 3=Verified（ACTION:VERIFY 晋升）
    CreatedAt     int64    `json:"createdAt"`
}
```

2.2.3 子循环状态（api/subLoopState）

Listing 3: internal/api/subloop.go:40-52

```
type subLoopState struct {
    Name, Goal, Status string   // running|done|failed|cancelled
    Turns  int
    LastReply, Progress string  // Progress: 模型每5轮写的摘要 + 后端机械兜底
    PivotCount int              // Progress 门掉头次数（上限2）
    DoneRejected bool           // DONE 驳回是否已用过（仅一次）
```

```
    StartedAt, EndedAt time.Time
    Error  string
}
// 常量: solve 循环 40 轮 / 40 分钟; recon 微循环 5 轮 / 6 分钟
// 并发上限: OURO_MAX_SUB_LOOPS（默认 2, benchmark 配 3）
```

2.3 动作协议（ACTION 文本协议，87 处引用）

模型回复中的文本行动作，后端逐行解析执行（ai_chat.go:553 `parseAndExecuteAIActions`）。完整语法：

```
ACTION:OBSERVE <url> [{"method":"POST","body":"...","headers":{...},"baseline":true
    }]
ACTION:BENCH list | start <code> | submit <code> <flag> | close <code> | hint <code>
ACTION:SPAWN <name> : <goal>          # 冒号分隔, name 禁止 <>
ACTION:COLLECT [<name>]
ACTION:BLACKBOARD <conclusion>        # 结论引用 obs-ID 自动回链
ACTION:VERIFY <obs-id> <expected-signal>
ACTION:EXECUTE CONFIRMED: <bash>      # CONFIRMED: 前缀=提示注入防线
ACTION:READ|WRITE|LIST|SEARCH|DELETE|RENAME <path...>
ACTION:REPORT <path> <markdown>
ACTION:GOAL|GOAL-DONE|PROMPT-SAVE|HARNESS|SWARM|AGENT|AGENT-RUN|AGENT-DONE|RETRY
```

解析管线（图 ??）：

1. `coalesceActionLines`（ai_chat.go:480）：括号平衡合并跨行 JSON spec（模型 markdown 习惯把 spec 放下一行——v6 曾因此 422 死循环两个容器）
2. 逐行前缀匹配分发（87 处 if-else 链）
3. 各分支执行（OBSERVE 过 scope 门 →ObserveHTTP→ 蒸馏 → 落盘；BENCH 直调平台；SPAWN 起 goroutine；EXECUTE 过 egress 门 →`bash -c`）
4. 结果以 ChatActionResult 追加到回复尾部

2.4 chat 主流程时序（ai_chat.go:96 handleAIChatPost）

1. 收 `{message, context?, maxTokens?}`；message 持久化到 ai-conversation.jsonl
2. 组装 system prompt：工作区摘要（mission/scope/黑板事实 top15/活跃执行）+ 20 条动作教学 + 自主性契约 8 条（autonomy.go，含第 7 条思维/动作分离、第 8 条探索阶段）
3. `GenerateProviderMessage`（ctx 60s 超时，瞬态网络错误 3s/9s 双重试）→ 模型回复
4. 回复持久化；`parseAndExecuteAIActions` 执行动作；结果 + 引用追加
5. 响应 `{message}` 返回

2.5 子循环生命周期（subloop.go:82/247/487）

```
SPAWN: 注册表登记+持久化 -> 黑板种子(相关结论top5+超图goal命中)
       + systemPrompt(角色+方法+PROGRESS契约+自主契约+种子) -> goroutine
循环: GenerateProviderMessage(CognitiveTier=1, MaxTokens=65536)
      -> 执行动作行(DONE前先执行!) -> PROGRESS提取/机械兜底
      -> IGR检测(5轮零新增->掉头注入,PivotCount<2)
      -> DONE驳回检查(无flag事实且未驳回过->驳回一次)
完成: 通知注入父会话(状态+进展+最后回复截断) + 注册表更新
```

2.6 driver 编排（Python，benchmark/）

三个 driver 形态并存（OURO_DRIVE_MODE 选择）：

- **parent**（drive_parent.py）：回合制对话驱动。每轮注入观测事实（平台对账 + 子循环状态 + 编排缺口模糊匹配 + 预算观测），失败感知（模型调用失败不推进轮次退避重发），idle≥2 干预，重复观测警告，产出通牒（4 轮零实质产出 → 三选一强制）
- **orchestrated**（drive_orchestrated.py，v24+ 默认）：硬流水线。20s 轮询：平台对账 → 缺口补位 start（easy 优先）→ 模型一行决策（maxTokens=96）→ 沉默则默认策略 SPAWN→ 收割（从 lastReply 正则提取 flag 直调 submit）
- SPAWN 走 chat API verbatim 回显（maxTokens=64）——架构跟自己说话

2.7 Docker 形态

python:3.12-slim + 静态 Go 二进制（20.8MB）+ tsec-benchmark SDK（离线 wheel）+ kb（79 技能 + 双索引）+ drivers。全部凭证 env 注入零落盘；`--restart=always`；workspace bootstrap 在 driver 内（providers.yaml 用 base_url_env/api_key_env 变量名引用）。

3 实测失败模式（FM-1 至 FM-5，含案例原文）

3.1 FM-1：分析瘫痪（未解决，最高优先级）

FM-1 现象与实测数据

v21：73 条消息、15+ 轮、idle=14。SPAWN 目标在 30+ 条消息中被反复起草、审议、重写，从未执行。

轮均输出：7,735–8,510 字符（v22 实测 4 轮）。动作行位置在第 1,507 字符（即长分析在前）。

案例原文（v20 msg#65，旗舰 glm-5.3）：

> "Let me write the goals: e1-01-solve: 解题任务 e1-01: 靶场地址...
> 【动作结果分析】... So: ACTION:OBSERVE {{env:...}} ..." （SPAWN 又被换成观测，循环继续）

案例原文（v22 尾部，对 close 职责的哲学审议）：

> "...But if sub-loop completes all flags, it could close too. But closing frees the active slot —that's fine actually, it lets..." （一轮 8.5K 字符耗尽，无动作）

修复尝试	版本/结果	失效原因
自主契约 #6：动作必须在场	v20 / 部分	动作出现但位于 1500+ 字符处；分析仍占 70%+
action-first：动作行开头	v22 / 部分	每轮有动作了，但 SPAWN 这类” 重动作” 仍被审议延迟
产出通牒：4 轮零产出 → 三选一	v21 / 失效	模型” 审议通牒合法性”——把干预当新约束去优化
token 硬帽 64/96	v26 / 仅管道有效	只约束 driver 自言自语场景；主对话 8K 帽下分析仍挤占
思维/动作分离提示	v25 / 弱	GLM 的 reasoning 与回复体不分离，提示无法强制

表 2: FM-1 五次修复尝试全部失效或部分失效

理论诊断：这不是提示词问题而是协议问题——文本协议下” 分析” 与” 动作” 共享同一输出流，模型没有独立的结构化动作通道（Anthropic tool_use 有）。**每次修复都被模型当作新的语义约束去” 合规审议”，产生新的分析对象**——修复本身成为瘫痪素材。

3.2 FM-2：格式执行失败（三代守卫，治标）

FM-2 失败案例时间线

v19： `ACTION:SPAWN <name> : <independent goal>` ——教学示例的尖括号被照抄执行。
v22： goal 含 `<flag>`（合法指代）被守卫误伤（守卫只放行了 `<flag 值 >` 中文形态）。
v22： goal 含 `{{env:BENCHMARK_BASE_URL}}` 又被守卫拦（env 占位符 ≠ 文档占位符）。
v25： verbatim 回显提示下模型输出 `SPAWN d-01 : ...`——丢了 `ACTION:` 前缀，解析失败。
JSON spec 中文引号： `invalid character 'æ'` ——智能引号混入 spec（已归一化修复）。
修复链：黑名单守卫 → 白名单 → 只查 name 位 →verbatim 回显。每代都发现新的误伤/漏伤形态。

3.3 FM-3：重复获取（信息经济学失衡）

v22 实测输入构成（会话记录统计）

- 同一列题 OBSERVE 发起 3 次（每次 26KB 响应进上下文）
- `ACTION:READ runtime/observations.jsonl` 读全量观测记录（2 万字级）
- 手写 python 解析题单 JSON body——而 `BENCH list` 蒸馏一行可得

量化：输入 ~60% 已见信息 / ~25% 系统提示 / ~15% 新证据；输出 ~70% 无行动分析 / ~20% 格式 / ~10% 动作。**有效密度** < 15%。

已修：重复观测警告（driver 检测同 URL ≥2 次 →” 禁止再观测”）；BENCH list 蒸馏加活跃题详情。残留：警告是劝不是禁；黑板/观测**没有查询接口**，READ 全文是唯一获取方式。

3.4 FM-4：指令二义性死锁

” 微任务教学”（任务要拆细！每个子循环几分钟完成）与题目天然整题目标冲突 → 模型审议” 我的 SPAWN 是否违反教学”（v20 实测原文：”The user’s spawn requests are full-challenge

outsourcing... That's against...")。已修：双许可（easy 可整题）。**教训：教学措辞是无约束边界的语义空间，模型会把它当硬约束优化。**

3.5 FM-5：环境层（已驯服）

网络抖动 ~40 次（provider TLS 超时/断连）→ 双层重试吸收；Docker Desktop 三次崩溃 → restart=always；agent 因 workspace 目录缺失死亡 → bootstrap 修复；pip 构建层失败 → 离线 wheel 根治；driver 900s 超时挂死 → 240s+12 重试。

3.6 成功侧记录（公平性）

v8 时代 flash 模型解出 e1-01/e1-02（各 250 分，含完整 submit 响应证据）；子循环多次展示高质量行为：识别 DVWA 并测默认凭据、设计反代绕过策略、PROGRESS 摘要纪律、黑板契约沉淀（" 容器只有 python3，所有子循环停止尝试 curl"——跨子循环知识共享真实发生）；BENCH 传送带被采用后全部秒级成功。

4 关键不对称与理论推导

核心发现：子循环质量 ≫ 父循环质量

- **子循环**（自治 goroutine，单一目标）：推理质量高、格式纪律好、黑板沉淀规范
- **父循环**（编排对话）：FM-1/2/3/4 全部集中于此
- **结论**：编排调度是当前 LLM 最不擅长的任务形态（在不确定中做节奏决策 + 格式纪律），侦察-解题反而是强项

4.1 三个结构性判断（v2 推导基础）

1. **决策-执行耦合是结构性错误**。文本协议下模型必须输出正确格式才能推进：认知优势被格式纪律拖累，架构可靠性被等待模型拖累。Anthropic 生产系统的隐含答案：tool_use 结构化调用（模型选工具与参数，协议层管格式）。我们的 ACTION: 文本协议在这点上落后一代。
2. **黑板有存储无检索**。模型只能 READ 全文 → 信息密度塌缩。缺谓词查询原语。
3. **委派不对称律**：*编排节奏归架构，委派质量归模型，执行格式归协议*——三者各自最优化，不再互相拖累。v25 的" 硬流水线 + 模型一行决策 + 默认兜底" 是此律的初步落地，但决策仍走文本协议（FM-2 残留）。

4.2 v2 蓝图（已推导未实现）

4.3 待验证假设（下个靶场窗口）

5 给外援的五个裁决问题

1. **FM-1 根治路线**（分析瘫痪）。备选：(a) 意图协议——结构化输出让分析无处分流；(b) 后端强截断——只取动作行丢弃其余（丢掉的推理是否是浪费？模型会不会因输出被弃而退化？）；(c) 双通

L5 意图层（新）	chat API 加 `responseMode:"intent"`:模型返回 `{"intent":"spawn","target":"e1-01","goal":"..."}` 结构化意图，后端直接执行。FM-2 整类消灭；FM-1 的分析无处分流（意图 JSON 里没有分析字段）
L3 查询黑板	`{"intent":"query","predicates":{"host":"10.0.179.210","kind":"conclusion"}}` → 命中事实摘要列表。信息密度 15%→60%+
L4 已落地	v25 形态保留：硬流水线节奏 + 模型意图决策 + 默认策略兜底

表 3: GateMesh v2 三项增量

假设	实验设计	指标
意图协议消灭格式失败	intent-mode vs ACTION-text 各跑 10 题	格式错误次数/百轮
查询黑板提升密度	QUERY vs READ 各 5 会话	输入新信息占比（会话统计）
委派不对称提升吞吐	v27 vs v28	首题时间、题/小时、token/题

道生成——先自由分析再单独生成动作（成本 ×2）；(d) 强制 interleaved thinking（若 provider 暴露独立思考通道——GLM 系目前不暴露给 API 消费者）。我们的倾向：*(a)+(d)* 长期、*(a)* 短期。求：业界在回复体/思考不分离模型上的第四条路？

2. **意图 schema 的严格度**。`spawn/observe/submit` 的意图 JSON 需要多严格？自由 `goal` 字段（自然语言）与结构化字段（target/tool/params）的边界？schema 过严会不会把 FM-4（二义性审议）转移到 schema 遵守上？求：先例与设计原则。

3. **父循环的存废**。数据支持" 子循环强、父循环弱"：是否彻底删除父循环对话，driver 全机械编排 + 模型只出现在子循环内？风险：丢失 lead agent 的跨题综合判断（Anthropic 强调委派质量决定一切——而委派质量来自 lead 的策略视野）。折中方案（当前 v25）：模型一行决策 + 架构兜底是否足够？求：裁决与依据。

4. **黑板查询原语的语义设计**。谓词富度（key=value / 全文 contains / 时间窗 / 证据等级过滤）？返回形态（摘要列表 vs 带回链的完整事实）？有无记忆系统检索接口的业界先例（Voyager 技能库 / MemGPT / 其他）？

5. **评测方法论**。33 场数据、11 个版本——但每版多变量同改（修复 + 部署 + 重启）。Anthropic 建议 20 query 起步小样本快迭代。我们的单变量隔离（只改一层）实验设计是否正确？样本量多少才足以分辨 FM-1 类修复的真假效果（我们的 5 次修复都" 部分生效"——是修复弱还是测量噪声）？

附录 A：关键代码坐标速查

chat 主流程	ai_chat.go:96 `handleAIChatPost`
跨行合并	ai_chat.go:480 `coalesceActionLines`
动作解析	ai_chat.go:553 `parseAndExecuteAIActions`（87 处 ACTION 分支）
观测原语	nativetools/http_observe.go:104 `ObserveHTTP`
Scope 门	api/observe_scope.go:21 `authorizeObserveTarget`
EXECUTE 门	api/egress_gate.go `checkExecutionEgress`
传送带	api/conveyor.go `executeBenchAction` + 五蒸馏器
子循环	api/subloop.go:82 (spawn) :247 (run) :487 (collect)
超图	api/hypergraph.go（检索 + 切片）
蒸馏	api/observe_distill.go `distillObservationForAI`
Evidence 门	api/evidence_gate.go `executeVerifyAction`
自主契约	api/autonomy.go（8 条，系统提示词注入）
driver×3	benchmark/drive_parent.py / drive_orchestrated.py / parallel_entry.py